Auto-fill Amount Based on Vehicle Category - Complete Explanation

📌 Overview

The system automatically calculates the price for a load based on:

- **Vehicle Category** (type, capacity, length)
- **Distance** between pickup and drop
- **Weight** of the goods
- **Trip Type** (one-way/return)
- **Trip Mode** (full truck/part load)

🎯 Real-Life Example Scenario

Scenario Details:

text

Customer: ABC Traders needs to transport goods

Pickup: Pune (18.5204° N, 73.8567° E)

Drop: Mumbai (19.0760° N, 72.8777° E)

Material: Electronic Goods

Weight: 8,500 kg (8.5 tons)

Vehicle Required: 16-ton Container Truck

Trip Type: One Way

Date: Weekday, 10:00 AM

📊 Step-by-Step Calculation Process

Step 1: Distance Calculation (Haversine Formula)

python

Pune: 18.5204° N, 73.8567° E

Mumbai: 19.0760° N, 72.8777° E

Formula: $d = 2R \times \arcsin(\sqrt{(\sin^2(\Delta\phi/2) + \cos\phi_1 \times \cos\phi_2 \times \sin^2(\Delta\lambda/2))})$

Calculation:

$\Delta\phi = 19.0760 - 18.5204 = 0.5556° = 0.0097$ radians

$\Delta\lambda = 72.8777 - 73.8567 = -0.979° = -0.0171$ radians

$a = \sin^2(0.0097/2) + \cos(18.5204°) \times \cos(19.0760°) \times \sin^2(-0.0171/2)$

$a = 0.0000235 + (0.9487 \times 0.9453 \times 0.0000731)$

$a = 0.0000235 + 0.0000655 = 0.000089$

$c = 2 \times \arcsin(\sqrt{0.000089}) = 2 \times 0.00943 = 0.01886$

Distance = 6371 × 0.01886 = 120.15 km ≈ 120 km

Step 2: Vehicle Category Selection

The system checks the VehicleCategory model:

```python
vehicle_category = VehicleCategory.objects.get(
    vehicle_type='container',
    capacity_ton=16.00,
    is_active=True
)

# Category Data:
# - name: "16 Ton Container"
# - base_rate_per_km: ₹30.00
# - base_distance_km: 240
# - base_charge: ₹7,000
```

Step 3: Find Applicable Pricing Slab

The system queries PricingSlab for matching slab:

```python
slab = PricingSlab.objects.filter(
    vehicle_category=vehicle_category,
    is_active=True,
    min_distance__lte=120,     # Distance <= 120 km
    max_distance__gte=120,      # Distance >= 120 km
    min_weight_ton__lte=8.5,    # Weight >= 8.5 tons
    max_weight_ton__gte=8.5     # Weight <= 8.5 tons
```

Found Slab:

- min_distance: 100 km

- max_distance: 200 km

- min_weight: 5 tons

- max_weight: 16 tons

- rate_per_km: ₹32/km (slightly higher due to weight)

- rate_per_ton_km: ₹2.5/ton/km

Step 4: Base Price Calculation

Option A: If Slab Found (Slab-based pricing)

python

weight_ton = 8.5 tons

distance = 120 km

Combined calculation (distance + weight)

base_price = (distance × rate_per_km) + (distance × rate_per_ton_km × weight_ton)

base_price = (120 × 32) + (120 × 2.5 × 8.5)

base_price = 3,840 + 2,550 = ₹6,390

Option B: If No Slab (Default formula)

python

base_price = (distance / base_distance_km) × base_charge

base_price = (120 / 240) × 7,000

base_price = 0.5 × 7,000 = ₹3,500

Note: Option B is used as fallback but we found a slab, so we use Option A

Step 5: Check Minimum Distance for Part Load

python

distance = 120 km (not < 5 km, so no multiplier)

If distance < 5 km: base_price × 1.5 (part_load_multiplier)

Step 6: Check Base Slab Minimum Charge

python

Check if this is base slab (1000 kg minimum)

if slab.is_base_slab and weight_kg <= 1000:

```
    if slab.base_charge and base_price < slab.base_charge:
        base_price = slab.base_charge

# Not applicable here (weight 8500 kg > 1000 kg)
```

Step 7: Apply Peak/Festival Multipliers

python

```
# Check current time
now = 10:00 AM, Tuesday

# Peak time check (8-11 AM on weekdays)
is_peak_time = True  # 10 AM on Tuesday
is_festival = False  # Not a festival

if is_festival:
    multiplier = slab.festival_multiplier  # 1.2 (20% extra)
elif is_peak_time:
    multiplier = slab.peak_multiplier      # 1.1 (10% extra)
else:
    multiplier = 1.0

# Apply multiplier
price_after_multiplier = base_price × multiplier
price_after_multiplier = 6,390 × 1.1 = ₹7,029
```

Step 8: Final Price Calculation

python

```
total_price = round(price_after_multiplier, 2) = ₹7,029

# Per ton rate for reference
per_ton_rate = total_price / weight_ton
per_ton_rate = 7,029 / 8.5 = ₹827/ton
```

📈 **Complete Price Breakdown**

```json
{
  "distance_km": 120,
  "base_price": 6390,
  "applicable_slab": {
    "id": 5,
    "min_distance": 100,
    "max_distance": 200,
    "rate_per_km": 32,
    "rate_per_ton_km": 2.5
  },
  "multiplier": 1.1,
  "multiplier_reason": "Peak Time",
  "total_price": 7029,
  "per_ton_rate": 827,
  "breakdown": {
    "distance": 120,
    "weight_tons": 8.5,
    "base_calculation": "Slab Based",
    "rate_applied": 32,
    "calculation_formula": "(120 × 32) + (120 × 2.5 × 8.5) × 1.1"
  }
}
```

🔄 How the System Flows

1. User Creates Load

```python
# Frontend sends:
POST /api/loads/
{
  "pickup_address": "Pune Station, Pune",
  "pickup_lat": 18.5204,
```

```
    "pickup_lng": 73.8567,
    "drop_address": "Dadar, Mumbai",
    "drop_lat": 19.0760,
    "drop_lng": 72.8777,
    "weight_kg": 8500,
    "required_vehicle_category": 3,  # 16-ton Container
    "trip_type": "one_way",
    "trip_mode": "full_truck"
}
```

2. Serializer Processes

python

```
# LoadCreateWithPriceSerializer.validate()
1. Calculates distance using PricingCalculator
2. Gets vehicle category from database
3. Calls calculate_estimated_price()
4. Creates dummy objects for calculation
5. Returns price breakdown
```

3. Pricing Calculator in Action

python

```
# PricingCalculator.calculate_price()
1. Calculate distance (120 km)
2. Find vehicle category
3. Find applicable slab
4. Calculate base price (₹6,390)
5. Apply multipliers (₹7,029)
6. Return detailed breakdown
```

4. Price Displayed to User

python

```
Response includes:
- Estimated Price: ₹7,029
- Price Breakdown showing each component
```

- Option to accept auto-calculated price or override with manual price

🧮 **Mathematical Formulas Summary**

1. Distance Formula (Haversine)

text

$d = 2R \times \arcsin(\sqrt{\sin^2(\Delta\phi/2) + \cos \phi 1 \times \cos \phi 2 \times \sin^2(\Delta\lambda/2)})$

where R = 6371 km (Earth's radius)

2. Base Price Formula

text

If slab found:

Base = $(D \times R_1) + (D \times R_2 \times W)$

where:

D = Distance (km)

R_1 = Rate per km

R_2 = Rate per ton per km

W = Weight in tons

If no slab:

Base = $(D / B) \times C$

where:

B = Base distance (240 km)

C = Base charge (₹7,000)

3. Multiplier Application

text

Final = Base × M

where M = 1.0 (normal), 1.1 (peak), 1.2 (festival)

4. Minimum Distance Rule

text

If D < 5 km and trip_mode = 'part_load':

Final = Base × 1.5

5. Base Slab Minimum

text

If weight ≤ 1000 kg and slab.is_base_slab:

Final = max(Base, slab.base_charge)

💡 Real Business Logic

Why This Pricing Structure?

1. **₹7,000 for 240 km** - Industry standard rate for full truck
2. **Per km rates** - Accounts for fuel and operational costs
3. **Weight factors** - Heavier loads consume more fuel
4. **Peak pricing** - Higher demand during business hours
5. **Minimum distance** - Covers fixed costs even for short trips

Business Rules Enforced

- **Minimum 1 km** - Can't book for extremely short distances
- **Part load premium** - 50% extra for loads < 5 km
- **Peak hours** - 10% extra (8-11 AM, 5-8 PM)
- **Festival pricing** - 20% extra during festivals
- **Wallet limit** - Only 10% of trip cost from wallet

🎯 Example Variations

Example 2: Short Distance Part Load

text

Distance: 3 km

Weight: 500 kg

Result: ₹(base × 1.5) due to part_load_multiplier

Example 3: Festival Time

text

Distance: 120 km

Weight: 8.5 tons

Time: Diwali week

Result: ₹(base × 1.2) festival_multiplier

Example 4: Heavy Load Long Distance

text

Distance: 500 km

Weight: 32 tons

Result: Uses different slab with lower per-km rate

This automated pricing ensures:

- **Fair pricing** based on actual costs
- **Dynamic adjustments** for peak times
- **Transparency** with full breakdown
- **Consistency** across all bookings